

Лекция 3. Эйлеровость графов. 2-SAT.
Алгоритм Дейкстры. Алгоритм A^*

1 Эйлеровость графа

Определение 1.1. Эйлеров путь — это путь в графе, проходящий по каждому ребру ровно один раз.

Определение 1.2. Эйлеров цикл в графе — это эйлеров путь, являющийся циклом.

Определение 1.3. Неориентированный связный граф называется эйлеровым, если в нём существует эйлеров цикл.

Определение 1.4. Неориентированный связный граф называется полуэйлеровым, если в нём содержится эйлеров путь, но не содержится эйлеров цикл.

Теорема 1.5 (критерий эйлеровости, б/д). Следующие утверждения эквивалентны:

1. Граф эйлеров.
2. Степени всех вершин чётные.
3. Рёбра графа можно разбить на непересекающиеся по рёбрам циклы.

Доказательство. $\boxed{1} \Rightarrow \boxed{2}$. Следует из чётности количества рёбер, входящих в путь и смежных с каждой отдельно рассматриваемой вершиной.

$\boxed{2} \Rightarrow \boxed{3}$. Стартуем из произвольной вершины, строим цикл пока он не замкнётся. Следующий шаг всегда можно сделать по чётности степеней. Удалить все рёбра цикла. Повторить.

$\boxed{3} \Rightarrow \boxed{1}$. Доказательство по индукции по количеству непересекающихся циклов.

База. Если цикл один, то это и есть эйлеров цикл.

Шаг. Пусть для графов, распадающихся на не более n циклов, утверждение верно. Рассмотрим граф с $(n + 1)$ циклом. Рассмотрим (и запишем) $(n + 1)$ -ый цикл:

$$v_1 e_1 v_2 e_2 \dots e_{k-1} v_k$$

Удалим его из графа. Новый граф распадается на m компонент связности, для каждой из которых выполнено предположение индукции. Следовательно, для каждой компоненты связности есть эйлеров цикл (рёбра всех циклов разных компонент связности попарно различны). Существует эйлеров цикл, в который входит вершина из $\{v_1, \dots, v_k\}$, поскольку исходный граф был связным. Возможны несколько случаев:

1. Компонент больше одной. Рассмотрим вершины u и w из разных компонент, связанных в исходном графе путём, следовательно, путь проходил через удалённые рёбра (то есть содержал одно или несколько рёбер из $\{e_1, \dots, e_{k-1}\}$). Значит, содержал и хотя бы одну из вершин $\{v_1, \dots, v_k\}$. Рассмотрим первое ребро, выходящее за пределы этой компоненты связности. Оно может соединяться только с вершиной v_i . Для каждой компоненты перечислим цикл, начиная с v_i . Получим цикл, проходящий по каждому ребру ровно один раз.
2. Компонента связности одна, то это эйлеров граф, содержащий все вершины исходного графа.

□

Постановка задачи. Найти эйлеров цикл в графе, который заведомо является эйлеровым.

Алгоритм 1.6 (Хирхольцера, поиск эйлерова цикла). Алгоритм строит цикл, используя DFS с удалением пройденных рёбер.

1. Запускаем DFS из произвольной вершины v .

2. На каждом шаге переходим по любому непосещённому ребру в соседнюю вершину и удаляем это ребро из графа.
3. Если из текущей вершины нет исходящих непосещённых рёбер, добавляем её в ответ и возвращаемся на шаг рекурсии назад.
4. Поскольку степени вершин чётны, мы не можем «застрять» в вершине, отличной от стартовой, пока не используем все инцидентные ей рёбра.
5. В результате вершины записываются в порядке завершения обхода. Полученная последовательность представляет эйлеров цикл, записанный в обратном порядке.

Пример 1.7. Рассмотрим граф с вершинами $\{1, 2, 3, 4, 5\}$. Между вершинами 2 и 3 существуют два кратных ребра. Запуск DFS из вершины 1 может пройти путь: $1 \rightarrow 2 \rightarrow 5 \rightarrow 3 \rightarrow 4 \rightarrow 1$. В ответ выписываются 1 и 4, в вершине 3 осталось неиспользованное ребро в вершину 2. Обход продолжается: $3 \rightarrow 2 \rightarrow 3$. Теперь рёбра закончились везде. Порядок записи в ответ: 1, 4, 3, 2, 3, 5, 2, 1. После разворота получаем эйлеров цикл: $1 \rightarrow 2 \rightarrow 5 \rightarrow 3 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$.

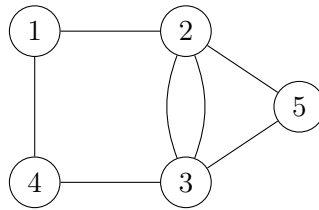


Рис. 1: Пример графа с эйлеровым циклом

Псевдокод.

```

1 void Dfs(Vertex v, Graph g) {
2     while (!g[v].Empty()) {
3         Vertex u = g[v].GetAnyVertex();
4         RemoveEdge(v, u, g);
5         Dfs(u, g);
6     }
7     cout << v << ' ';
8 }

```

Теорема 1.8 (корректность). *Представленный алгоритм находит эйлеров цикл в связном графе с чётными степенями вершин.*

Доказательство. Поскольку степени всех вершин чётны, при входе в любую вершину по неиспользованному ребру всегда найдётся неиспользованное ребро для выхода, за исключением случая, когда мы вернулись в стартовую вершину и использовали все рёбра графа. Следовательно, обход не прервётся преждевременно. Каждое ребро удаляется сразу после прохождения. Алгоритм завершается, когда все рёбра удалены. Следовательно, последовательность содержит ровно $|E| + 1$ вершин. Каждая соседняя пара в последовательности соединена ребром, и каждое ребро использовано ровно один раз. Так как граф связан, полученная последовательность образует единый цикл, проходящий по всем рёбрам. $\square$

Теорема 1.9 (б/д). *Эйлеров путь в графе существует, если граф связан и количество вершин с нечётной степенью не превосходит 2.*

Доказательство. Разобьём доказательство на три случая по количеству вершин нечётной степени.
Случай 1: 0 вершин нечётной степени. В случае, когда вершин с нечётной степенью нет, граф связан и степени всех вершин чётные. По критерию эйлеровости в графе есть эйлеров цикл. Поскольку эйлеров цикл является эйлеровым путём, следовательно, в графе существует эйлеров путь.

Случай 2: 1 вершина нечётной степени. По лемме о рукопожатиях в графе не может быть ровно одной вершины с нечётной степенью.

Случай 3: 2 вершины нечётной степени. Пусть u и v — единственные вершины с нечётной степенью. Добавим в граф новое фиктивное ребро $e^* = (u, v)$. В полученном графе G' все вершины имеют чётную степень, и он связан. По первому случаю в графе G' существует эйлеров цикл C . Удалим ребро e^* из цикла C . Разрыв цикла превращает его в путь, который начинается в одной из вершин $\{u, v\}$ и заканчивается в другой, проходя по всем рёбрам исходного графа G ровно один раз. Следовательно, в графе G существует эйлеров путь. $\square$

Постановка задачи. Дан связный неориентированный граф. Требуется найти в нём эйлеров путь.

Алгоритм 1.10 (поиск эйлерова пути). Проверим, что в графе существует эйлеров путь. Для этого в графе должно быть не более 2 вершин с нечётной степенью. Если вершин с нечётной степенью нет, то граф эйлеров. Найдём в нём эйлеров цикл, он и будет эйлеровым путём. Если вершин с нечётной степенью две, то соединим эти две вершины ребром и найдём эйлеров цикл в полученном графе. Удалим из цикла добавленное ребро и получим эйлеров путь в исходном графе.

2 Общая постановка задачи SAT

Постановка задачи SAT (*boolean satisfiability problem*). Дана булева формула Φ , заданная над множеством $X = \{x_1, x_2, \dots, x_n\}$. Представляет собой конъюнкцию одного или нескольких дизъюнктов, где каждый дизъюнкт состоит из литералов (переменных x_i или их отрицания $\bar{x}_i$) вида $(l_1 \vee l_2 \vee \dots \vee l_k)$, где l_j — литералы. Формула в КНФ: $\Phi = C_1 \wedge C_2 \wedge \dots \wedge C_m$, где C_j — дизъюнкты. Требуется ответить на вопрос, существует ли набор значений $\{v_1, \dots, v_n\}$, где $v_i \in \{0, 1\}$, такой, что $\Phi(v_1, \dots, v_n) = 1$. Если ответ положительный, предоставить сам набор.

Пример 2.1. $\Phi = (x_1 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee x_3) \wedge (x_2 \vee \bar{x}_3)$. Если положить $x_1 = 1, x_2 = 1, x_3 = 1$, то получим $(1 \vee 0) \wedge (0 \vee 1) \wedge (1 \vee 0) = 1 \wedge 1 \wedge 1 = 1$. Следовательно, формула выполнима, например, на наборе $\{1, 1, 1\}$.

Теорема 2.2 (Кука — Левина, б/д). Задача SAT является NP-полной.

Доказательство. Простое доказательство на 6 страниц уровня советских ясель $\square$

Замечание 2.3. В 1970-м году в статье Стивена Кука был впервые введён термин «NP-полная задача», и задача SAT была первой задачей, для которой доказывалось это свойство. Чаще всего для доказательства NP-полноты некоторой задачи приводится полиномиальная сводимость задачи SAT к данной задаче, возможно в несколько шагов, то есть с использованием нескольких промежуточных задач.

Теорема 2.4 (3-выполнимости, б/д). Любая задача SAT может быть полиномиально преобразована в задачу 3-SAT, где каждый дизъюнкт содержит ровно 3 литерала.

Доказательство. 8 страница того же самого доказательства $\square$

3 Задача 2-SAT

Определение 3.1. 2-SAT — это частный случай задачи SAT, где каждый дизъюнкт содержит ровно 2 литерала.

Алгоритм 3.2 (решение задачи 2-SAT). Решение задачи 2-SAT основано на сведении задачи к поиску компонент сильной связности в ориентированном графе. В отличие от общей задачи SAT, 2-SAT решается за линейное время $O(|V| + |E|)$, где $|V|$ — количество переменных, а $|E|$ — количество дизъюнктов. Построим ориентированный граф, вершинами которого будут литералы (переменные и их отрицания). Для каждого дизъюнкта вида $x \vee y$ заметим следующее тождество: $x \vee y \equiv (\bar{x} \rightarrow y) \wedge (\bar{y} \rightarrow x)$. Добавим в граф два направленных ребра:

1. $\bar{x} \rightarrow y$.

2. $\bar{y} \rightarrow x$.

Если дизъюнкт состоит из одного литерала ($a \vee a$), то добавляется ребро $\bar{a} \rightarrow a$.

Найдём компоненты сильной связности в полученном графе, например, алгоритмом Косарайю. Две вершины будут находиться в одной компоненте сильной связности тогда и только тогда, когда достижимы друг из друга по направленным рёбрам.

Если вершины x_i и $\bar{x}_i$ принадлежат одной и той же компоненте сильной связности, то решения не существует, поскольку $x_i \rightarrow \dots \rightarrow \bar{x}_i$ и $\bar{x}_i \rightarrow \dots \rightarrow x_i$. То есть $(x_i \rightarrow \bar{x}_i) \wedge (\bar{x}_i \rightarrow x_i)$, что является тождественным 0, а значит, все импликации не могут одновременно давать 1.

Если для всех переменных x_i и $\bar{x}_i$ лежат в разных компонентах, то решение существует.

Если задача выполнима, построим топологическую сортировку компонент сильной связности. Для каждой переменной x_i сравним номера компонент сильной связности, содержащих x_i и $\bar{x}_i$: если компонента с x_i находится раньше в топологическом порядке, то присвоим $x_i = 0$, иначе $x_i = 1$. То есть если x_i раньше, то в конденсации нет пути от $\bar{x}_i$ к x_i . Следовательно, выбор $x_i = 0$ не приведёт к логическому противоречию через цепочку импликаций.

Таким образом, алгоритм проверяет выполнимость задачи и находит одно из возможных решений, при этом строит граф на $2|V|$ вершинах и $2|E|$ рёбрах, конденсирует его и топологически сортирует. Сложность решения $O(|V| + |E|)$.

Пример 3.3. $\Phi = (\bar{x}_2 \vee x_4) \wedge (x_2 \vee x_3) \wedge (\bar{x}_1 \vee x_0) \wedge (x_2 \vee \bar{x}_0) \wedge (\bar{x}_0 \vee \bar{x}_3) \wedge (\bar{x}_3 \vee \bar{x}_2) \wedge (x_0 \vee \bar{x}_2) \wedge (\bar{x}_3 \vee x_0)$.
Возможное решение: $\{1, 0, 1, 0, 1\}$.

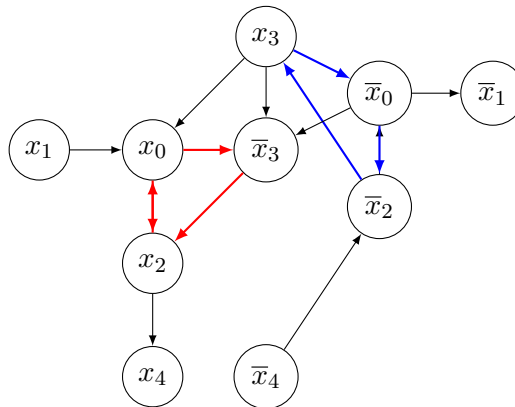


Рис. 2: Визуализация графа 2-SAT

4 Алгоритм Дейкстры

Напоминание. $\text{dist}(s, v)$ — длина кратчайшего пути из s в v , а $d[v]$ — оценка сверху длины кратчайшего пути $s \rightarrow v$. По определению в любой момент $\forall v \in V \hookrightarrow \text{dist}(s, v) \leq d[v]$. Часто используется значение $d[v] = +\infty$ для вершин, ещё не достигнутых из s .

Постановка задачи. Дан взвешенный граф (ориентированный или неориентированный) $G = (V, E, w)$ с неотрицательными весами рёбер $w : E \rightarrow K = \mathbb{R}^+$. Требуется найти расстояния $\text{dist}(s, v)$ для всех $v \in V$ (задача кратчайших путей от одного источника).

Подведение. Если все веса принадлежат множеству $\{0, \dots, k\}$ с небольшим целым k , то можно применять модификацию BFS (0-k BFS) с асимптотикой $O(k \cdot |V| + |E|)$. При больших k или при произвольных вещественных неотрицательных весах этот подход не даёт приемлемой оценки, и используют алгоритм Дейкстры. Он решает эту задачу, используя наблюдение, что из-за неотрицательности весов вдоль любого пути $v_1 v_2 \dots v_k$ расстояние от v_1 до v_i не больше, чем расстояние от v_1 до v_{i+1} .

Алгоритм 4.1 (Дейкстры). Пусть множество M — множество вершин, для которых кратчайшее расстояние dist уже вычислено. Изначально $M = \{s\}$, $d[s] = 0$, $d[i] = \infty \forall i \neq s$.

1. Среди всех вершин $u \notin M$ найдём такую, что текущее расстояние от M до неё минимально (заметьте, что v смежна с M), то есть формально

$$u = \arg \min_{t \in S} d[t]$$

2. Добавим в множество M вершину u , то есть $M = M \cup \{u\}$, присвоим $\text{dist}(s, u) = d[u]$.
3. Для рёбер вида $(u \rightarrow v)$ проведём релаксацию, то есть обновим $d[v] = \min(d[v], \text{dist}(s, u) + w(u, v))$.
4. Повторять, пока $M \neq V$.

Псевдокод. Реализация на массиве, асимптотика $O(|V|^2)$.

```

1 vector<int> d(vertices_count);
2 vector<bool> used(vertices_count);
3 for (int i = 0; i < vertices_count; ++i) {
4     d[i] = inf;
5     used[i] = false;
6 }
7 d[s] = 0;
8 for (int i = 0; i < vertices_count; ++i) {
9     int u = -1;
10    for (int t = 0; t < vertices_count; ++t) {
11        if (!used[t] && (u == -1 || d[t] < d[u])) {
12            u = t;
13        }
14    }
15    if (d[u] == inf) {
16        break;
17    }
18    used[u] = true;
19    for (auto [v, w] : graph[u]) {
20        d[v] = min(d[v], d[u] + w);
21    }
22 }
```

Псевдокод. Реализация на двоичной куче, асимптотика $O((|V| + |E|) \log |V|)$.

```

1 vector<int> d(vertices_count, inf);
2 BinaryHeap<pair<int, int>> q;
3 d[s] = 0;
4 q.emplace(0, s);
5 while (!q.empty()) {
6     auto [w, u] = q.top();
7     q.pop();
8     if (w != d[u]) {
9         continue;
10    }
11    for (auto [v, new_w] : graph[u]) {
12        if (d[u] + w < d[v]) {
13            d[v] = d[u] + w;
14            q.emplace(d[v], v);
15        }
16    }
17 }

```

Пример 4.2. Ниже изображена визуализация алгоритма Дейкстры. В вершине первое число — это её номер, а второе — текущая оценка на расстояние от вершины $s = 1$ до неё $d[u]$. Синим изображено множество вершин M и все рёбра между этими вершинами, красным — вершина v , до которой расстояние от M минимально, зелёным — те рёбра из вершины v , вдоль которых произошла успешная релаксация.

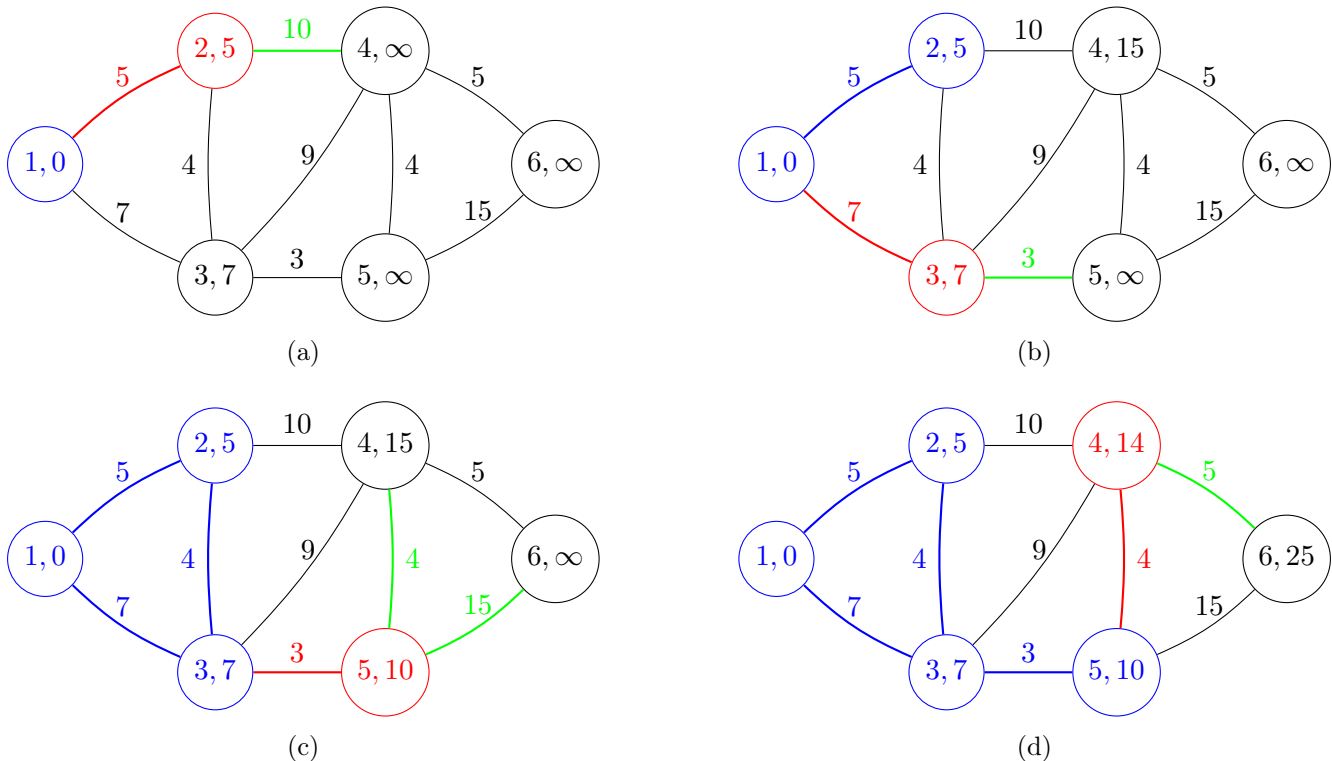


Рис. 3: Визуализация работы алгоритма Дейкстры

Теорема 4.3 (корректность). На момент добавления вершины v в M верно, что $\text{dist}(s, v) = d[v]$, то есть инвариант множества M сохраняется, и текущая оценка расстояния совпадает с истинным кратчайшим расстоянием.

Доказательство. Доказательство от противного. Пусть v — первая вершина, которая была добавлена в M , для которой $d[v] > \text{dist}(s, v)$. Пусть P — кратчайший путь из s в v , то есть $w(P) = \text{dist}(s, v)$. Поскольку перед добавлением v в множество $s \in M$, а $v \notin M$, на пути P существует первое ребро (x, y) , такое что: $x \in M$ — $y \notin M$.

Такое ребро найдётся, поскольку путь начинается в M , а заканчивается вне M . Поскольку $x \in M$, то по предположению (вершина v — первая, для которой нарушается инвариант) $d[x] = \text{dist}(s, x)$. Когда x добавлялась в M , алгоритм выполнил релаксацию всех исходящих из неё рёбер, в том числе (x, y) . Следовательно, $d[y] \leq d[x] + w(x, y) = \text{dist}(s, x) + w(x, y)$. Но правая часть — это длина части пути P от s до y (так как префикс кратчайшего пути также является кратчайшим путём), следовательно, $d[y] \leq \text{dist}(s, y) \leq \text{dist}(s, v)$, где последнее неравенство следует из неотрицательности весов (подпут к кратчайшему пути не может быть длиннее самого пути). В момент выбора вершины v для добавления в M вершина y ещё не была в M (по построению), но алгоритм выбирает вершину с минимальным значением $d[i]$ среди всех необработанных. Тогда $d[v] \leq d[y]$. Комбинируя последние два неравенства, получаем $d[v] \leq d[y] \leq \text{dist}(s, v)$. Следовательно, $d[v] = \text{dist}(s, v)$ — противоречие. Предположение о существовании такой вершины v ложно. Следовательно, для любой вершины v , добавляемой в M , выполняется $d[v] = \text{dist}(s, v)$. $\square$

Теорема 4.4 (время работы). *Алгоритм Дейкстры делает $O(|E|)$ релаксаций.*

Доказательство. Релаксация ребра (u, v) выполняется только в момент, когда вершина u добавляется в множество M . После этого вершина u больше никогда не обрабатывается. Таким образом, для каждого ребра (u, v) произойдёт не более двух релаксаций: со стороны u и со стороны v . $\square$

Замечание 4.5. *Получение вершины с минимальной оценкой d из $V \setminus M$ происходит $O(|V|)$ раз, а всего $O(|E|)$ релаксаций. В зависимости от того, как поддерживать получение вершины с минимальной оценкой d , время алгоритма будет разное:*

Контейнер вершин	Релаксация	Извлечение	Итого
Массив	$O(1)$	$O(V)$	$O(V ^2)$
Двоичная куча	$O(\log V)$	$O(\log V)$	$O((V + E) \log V)$
Фибоначчиева куча	$O^*(1)$	$O(\log V)$	$O(E + V \log V)$

Доказательство. В случае массива происходит $O(|V|)$ извлечений минимума, каждый за $O(|V|)$, всего $O(|V|^2)$, релаксаций всего $O(|E|)$, каждая из которых происходит за $O(1)$. Всего $O(|E|)$. Поскольку $|E| = O(|V|^2)$, сложность алгоритма равна $O(|V|^2)$.

В случае двоичной кучи на извлечение уходит $O(|V|) \cdot O(\log |V|) = O(|V| \log |V|)$ времени, на релаксацию $O(\log |V|) \cdot O(|E|) = O(|E| \log |V|)$, всего $O((|V| + |E|) \log |V|)$. В типичных задачах выражают как $O(|E| \log |V|)$ для графов с $|E| \geq |V|$. Фибоначчиева куча будет подробно разобрана в третьем семестре. $\square$

Замечание 4.6. *Алгоритм Дейкстры — достаточно мощный инструмент, который покрывает большинство задач, но у него есть два существенных недостатка.*

1. Ослабим условие задачи и потребуем найти расстояние от вершины s до одной вершины t . К сожалению, алгоритм Дейкстры не упрощается в этой задаче, время работы всё равно будет пропорционально $|V|$. Например, если бы хотели написать навигатор, который строит маршрут из точки A на карте до точки B , перемещаясь только по существующим дорогам, алгоритм Дейкстры не завершил бы поиск гарантированно кратчайшего пути, пока вершина B не оказалась бы в множестве M , что не гарантируется быстрее, чем за $O(|V|^2)$ или $O(|E| \log |V|)$. Таким образом, даже если путь $A \rightarrow B$ не очень длинный, алгоритм Дейкстры всё равно может исследовать большую часть графа за время, пропорциональное его размеру, что неприемлемо для систем навигации в реальном времени на больших картах.

2. Невозможность работы с рёбрами отрицательного веса.

5 Алгоритм A*

Подведение. Алгоритм A* (A-star, A-звезда, A со звёздочкой) частично решает проблему низкой производительности алгоритма Дейкстры при поиске пути до одной конкретной вершины t , пользуясь эвристическими знаниями об особенностях задачи. Например, в навигаторе кажется логичным искать кратчайший маршрут из A в B , приоритезируя направления, которые геометрически приближают к B (используя, например, евклидово расстояние как оценку остаточного пути). Алгоритм A* формализует эту интуицию.

Определение 5.1 (формальное). Эвристика — это практический метод или правило, которое не гарантирует нахождение оптимального или строго доказанного решения, но позволяет найти достаточно хорошее решение за приемлемое время, особенно в ситуациях, когда полный перебор всех вариантов невозможен из-за ограничений по времени или вычислительным ресурсам.

Определение 5.2 (неформальное). Эвристика — это какая-либо идея, основанная на свойствах конкретной задачи.

Определение 5.3. Эвристический алгоритм — это алгоритм, использующий в своей работе эвристики.

Постановка задачи. Дан взвешенный граф (ориентированный или неориентированный) с весовой функцией $w : E \rightarrow K = \mathbb{R}^+$, две вершины s и t и эвристическая функция $h : V \rightarrow \mathbb{R}^+$, имеющая смысл оценки на $\text{dist}(v, t)$. Требуется найти кратчайшее расстояние от s до t — $\text{dist}(s, t)$.

Алгоритм 5.4. Алгоритм A* является модификацией алгоритма Дейкстры. Главное отличие заключается в критерии выбора следующей вершины для обработки: вместо минимизации текущего расстояния $d[v]$, выбирается вершина с минимальным значением оценочной функции $f(x) = d[v] + h(v)$, где $d[v]$ — известное на данный момент расстояние от s до v , а $h(v)$ — эвристическая оценка расстояния от v до t .

Псевдокод.

```

1 d[s] = 0;
2 BinaryHeap q;
3 q.emplace(d[s] + h(s), d[s], s); // sort by d[v] + h[v]
4 while (!q.empty()) {
5     auto [hh, w, u] = q.top();
6     q.pop();
7     if (d[u] != w) {
8         continue;
9     }
10    for (auto [v, new_w] : g[u]) {
11        if (d[v] > w + new_w) {
12            d[v] = w + new_w;
13            q.emplace(d[v] + h(v), d[v], v);
14        }
15    }
16 }
```

Определение 5.5. Эвристика $h(v)$ допустима, если $\forall v \in V \hookrightarrow h(v) \leq \text{dist}(v, t)$.

Замечание 5.6. $h(t) \leq \text{dist}(t, t) = 0 \Rightarrow h(t) = 0$.

Определение 5.7. Эвристика $h(v)$ монотонна, если $\forall (u, v) \in E \hookrightarrow h(u) - h(v) \leq w(u, v)$ и $h(t) = 0$.

Теорема 5.8. Любая монотонная эвристика допустима.

Доказательство. Пусть h — монотонная эвристика. Покажем, что $\forall v \in V \hookrightarrow h(v) \leq \text{dist}(v, t)$. Фиксируем произвольную вершину $v \in V$. Если пути из v в t не существует, то $\text{dist}(v, t) = +\infty$, и неравенство $h(v) \leq +\infty$ выполнено. Если путь существует, то существует и кратчайший путь P из v в t :

$$P = v_0, v_1, \dots, v_k, \quad v_0 = v, \quad v_k = t$$

Длина этого пути равна $\text{dist}(v, t) = \sum_{i=0}^{k-1} w(v_i, v_{i+1})$. Для каждого ребра (v_i, v_{i+1}) по условию монотонности эвристики выполнено: $h(v_i) - h(v_{i+1}) \leq w(v_i, v_{i+1})$. Получаем систему из $(k-1)$ неравенств:

$$\begin{cases} h(v_0) - h(v_1) \leq w(v_0, v_1) \\ h(v_1) - h(v_2) \leq w(v_1, v_2) \\ \vdots \\ h(v_{k-1}) - h(v_k) \leq w(v_{k-1}, v_k) \end{cases}$$

Сложим все неравенства. В левой части получим

$$[h(v_0) - h(v_1)] + [h(v_1) - h(v_2)] + \dots + [h(v_{k-1}) - h(v_k)] = h(v_0) - h(v_k)$$

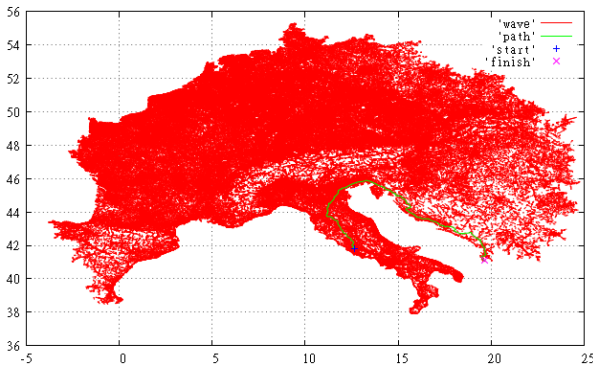
А в правой части сумма весов рёбер, то есть длина пути P : $\sum_{i=0}^{k-1} w(v_i, v_{i+1}) = \text{dist}(v, t)$. Получаем

$$h(v_0) - h(v_k) = h(v) - h(t) \leq \text{dist}(v, t)$$

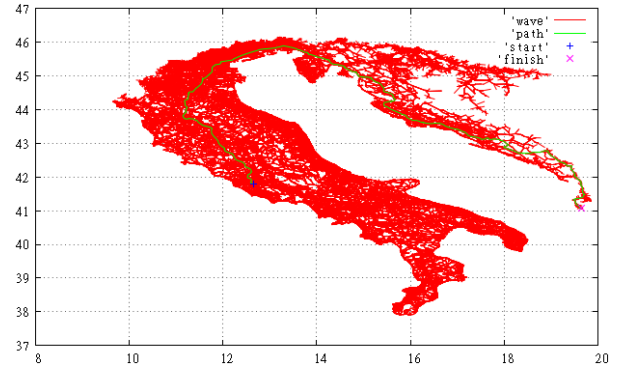
По определению монотонной эвристики $h(t) = 0$. Следовательно $h(v) \leq \text{dist}(v, t)$. Получили определение допустимой эвристики — что и требовалось доказать. $\square$

Теорема 5.9 (б/д). В случае монотонной эвристики алгоритм A^* работает не хуже алгоритма Дейкстры. В случае допустимой эвристики алгоритм A^* работает долго, но гарантируется, что найденный ответ будет верным.

Пример 5.10. Требуется найти кратчайший путь из одного города в Италии до одного из городов Албании. В качестве эвристики используется евклидово расстояние.



(а) Алгоритм Дейкстры



(б) Алгоритм A^*

Рис. 4: Сравнение количества просмотренных дорог алгоритмами Дейкстры и A^*